ISO/IEC JTC1 SC22 WG21 P2066R3
Authors: Hans Boehm, Victor Luchangco, Jens Maurer, Michael L. Scott, Michael Spear, and Michael Wong
Reply to: Jens Maurer <Jens.Maurer@gmx.net>
Target audience: EWG, LEWG
2020-09-15

P2066R3: Suggested draft TS for C++ Extensions for Minimal Transactional Memory

Introduction

This paper presents suggested wording for a future Technical Specification on a variant of Transactional Memory. See P1875R0 "Transactional Memory Lite Support in C++" for discussion.

Changes in R1 since R0

- incorporated feedback from SG1 in Prague

Changes in R2 since R1

- turned "atomic" into a context-sensitive keyword
- added questions for TS deployment

Changes in R3 since R2

- EWG feedback: extend definition of full-expression in [intro.execution], add note about parallel execution of transactions, constant evaluations in atomic blocks are irrelevant, use "atomic do" as the introducer keywords.
- Explicitly list standard library functions required to be supported in an atomic block

Questions for TS deployment

- Can we strengthen some of the "implementation-defined" parts of [stmt.tx]? In which way?
- What is the facility used for in practice?
- Are there problems that might benefit from transactions, but cannot be solved with the facilities presented?
- Do implementations provide additional facilities for transactions (possibly with extra syntax) beyond those presented?
- Is it vital for the implementation that the definitions of all functions invoked within a transaction are visible?
- Are there functions in the standard library that would be useful inside a transaction, but do not work yet?

Wording changes

These wording changes are relative to the current C++ Working Draft, N4849.

In 5.10 [lex.name], add **`atomic`** to table 4 [tab:lex.name.special].

Change in 6.9.1 [intro.execution] paragraph 5:

> A *full-expression* is
>
> - ...
> - an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.7.7) whose lifetime has not been extended, ~~or~~
> - **the start and the end of an atomic block (8.8 [stmt.tx]), or**
> - an expression that is not a subexpression of another expression and that is not otherwise part of a full-expression.

Change in 6.9.2.1 [intro.races] paragraph 6:

> **Atomic blocks as well as** ~~Certain~~ **certain** library calls **may** *synchronize with* other **atomic blocks and** library calls performed by another thread.

Add a new paragraph after 6.9.2.1 [intro.races] paragraph 20:

> **An atomic block that is not dynamically nested within another atomic block is termed a transaction. [Note: Due to syntactic constraints, blocks cannot overlap unless one is nested within the other.] There is a global total order of execution for all transactions. If, in that total order, a transaction T1 is ordered before a transaction T2, then**
>
> - **no evaluation in T2 happens before any evaluation in T1 and**
> - **if T1 and T2 perform conflicting expression evaluations, then the end of T1 synchronizes with the start of T2.**
>
> **[Note: If the evaluations in T1 and T2 do not conflict, they might be executed concurrently. -- end note]**
>
> Two actions are *potentially concurrent* if ...

Change in 6.9.2.1 [intro.races] paragraph 21:

> ... [Note: It can be shown that programs that correctly use mutexes**, atomic blocks,** and `memory_order::seq_cst` operations to prevent all data races and use no other synchronization operations behave as if the operations executed by their constituent threads were simply interleaved, with each value computation of an object being taken from the last side effect on that object in that interleaving. This is normally referred to as "sequential consistency". ...

Add a new paragraph after 6.9.2.1 [intro.races] paragraph 21:

> **[Note: The following holds for a data-race-free program: If the start of an atomic block T is sequenced before an evaluation A, A is sequenced before the end of T, A strongly happens before some evaluation B, and B is not sequenced before the end of T, then the end of T strongly happens before B. If an evaluation C strongly happens before that evaluation A and C is not sequenced after the start of T, then C strongly happens before the start of T. These properties in turn imply that in any simple interleaved (sequentially consistent) execution, the operations of each atomic block appear to be contiguous in the interleaving. -- end note]**

Change in 6.9.2.2 [intro.progress] paragraph 1:

> ~~The implementation may assume that any thread will eventually do~~ **An *inter-thread side effect* is** one of the following:
>
> - a call to a library I/O function,

- an access through a volatile glvalue, or
- a synchronization operation or an atomic operation.

The implementation may assume that any thread will eventually terminate or evaluate an inter-thread side effect. [Note: This is intended to allow compiler transformations such as removal of empty loops, even when termination cannot be proven. — end note]

Add a production to the grammar in 8.1 [expr.pre]:

```
statement:
      labeled-statement
      attribute-specifier-seq_opt expression-statement
      attribute-specifier-seq_opt compound-statement
      attribute-specifier-seq_opt selection-statement
      attribute-specifier-seq_opt iteration-statement
      attribute-specifier-seq_opt jump-statement
      declaration-statement
      attribute-specifier-seq_opt try-block
      atomic-statement
```

Add a new subclause before 8.8 [stmt.dcl]:

8.8 Atomic statement [stmt.tx]

```
atomic-statement:
     atomic do compound-statement
```

An atomic statement is also called an atomic block.

The start of the atomic block is immediately before the opening `{` of the *compound-statement*. The end of the atomic block is immediately after the closing `}` of the *compound-statement*. [Note: Thus, variables with automatic storage duration declared in the *compound-statement* are destroyed prior to reaching the end of the atomic block; see 8.7 [stmt.jump]. -- end note]

If the execution of an atomic block evaluates an inter-thread side effect (6.9.2.2 [intro.progress]), the behavior is undefined.

If the execution of an atomic block evaluates any of the following outside of a manifestly constant-evaluated context (7.7 [expr.const]), the behavior is implementation-defined:

- **an *asm-declaration* (9.10 [dcl.asm]);**
- **an invocation of a function other than an inline function with a reachable definition;**
- **a virtual function call (7.6.1.2 [expr.call]);**
- **a function call whose *postfix-expression* does not name a function (12.4.1.1 [over.match.call]);**
- **a *throw-expression* (7.6.18 [expr.throw]);**
- **a `co_await` expression (7.6.2.3 [expr.await]), a *yield-expression* (7.6.17 [expr.yield]), or a `co_return` statement (8.7.4 [stmt.return.coroutine]);**
- **dynamic initialization of a block-scope variable with static storage duration; or**
- **dynamic initialization of a variable with thread storage duration.**

[Note: Some functions in the standard library can be used in an atomic block (16.4.6.17 [atomic.use]).] [Note: The implementation may define that the behavior is undefined in some or all of the cases above.]

A goto or switch statement shall not be used to transfer control into an atomic block.

[Example:

```
int f()
{
  static int i = 0;
  atomic do {
    ++i;
    return i;
  }
}
```

Each invocation of f (even when called from several threads simultaneously) retrieves a unique value (ignoring overflow). -- end example]

Add 16.4.6.17 [atomic.use]:

16.4.6.17 Functions usable in an atomic block [atomic.use]

The following library functions can be used in an atomic block (8.8 [stmt.tx]):

- **`memset`, `memcpy`, `memmove` (21.5.3 [cstring.syn])**
- **`move` and `forward`**
- **`swap` and `exchange` for fundamental types ([basic.fundamental])**
- **all member functions of `array` (22.3.7 [array]) if the required operations on `T` are usable in an atomic block**
- **`list` (22.3.10 [list]) and `vector` (22.3.11 [vector]) access and iteration if the required operations on `T` are usable in an atomic block**
- **Metaprogramming and type traits ([meta])**
- **Compile-time rational arithmetic ([ratio])**
- **all member functions of `numeric_limits` ([numeric.limits])**
- **non-allocating forms of allocation and deallocation functions [new.delete.placement]**